

CS8.401: PRINCIPLES OF INFORMATION SECURITY

Programming Assignments

Comprehensive Specification Explanation

Contents

1	Prelude: The Cryptographic Universe and the Great Clique	3
1.1	Motivation	3
1.2	The Cryptographic Primitive Hierarchy	3
1.3	Note	3
1.4	Concrete Instantiations	7
1.5	The No-Library Rule	7
1.6	Security Notions Roadmap	8
2	Part 0: The Minicrypt Clique Web Explorer (Interactive Component)	9
2.1	The Core Problem the App Must Solve	9
2.2	Layout Specification	10
2.3	The Under-the-Hood Requirement (Critical)	11
2.4	Supported Reductions and the Routing Table	11
2.5	PA #0 — What You Must Implement	12
2.6	PA #1 — One-Way Functions & Pseudorandom Generators	14
2.7	PA #2 — Pseudorandom Functions via GGM Tree	15
2.8	PA #3 — CPA-Secure Symmetric Encryption	17
2.9	PA #4 — Modes of Operation	19
2.10	PA #5 — Message Authentication Codes (MACs)	21
2.11	PA #6 — CCA-Secure Symmetric Encryption	22
3	Part II: Hashing and Data Integrity	25
3.1	PA #7 — Merkle-Damgård Transform	25
3.2	PA #8 — DLP-Based Collision-Resistant Hash Function	27
3.3	PA #9 — Birthday Attack (Collision Finding)	29
3.4	PA #10 — HMAC and HMAC-Based CCA-Secure Encryption	30

4	Part III: Public-Key Cryptography (Cryptomania)	34
4.1	PA #11 — Diffie-Hellman Key Exchange (SKE)	34
4.2	PA #12 — Textbook RSA	35
4.3	PA #13 — Miller-Rabin Primality Testing	37
4.4	PA #14 — Chinese Remainder Theorem & Breaking Textbook RSA	39
4.5	PA #15 — Digital Signatures	41
4.6	PA #16 — ElGamal Public-Key Cryptosystem	43
4.7	PA #17 — CCA-Secure PKC	44
5	Part IV: Secure Multi-Party Computation (MPC)	46
5.1	PA #18 — Oblivious Transfer (OT)	46
5.2	PA #19 — Secure AND Gate	48
5.3	PA #20 — All 2-Party Secure Computation (Yao / GMW)	49
6	Summary: The Full Reduction Chain	52

1 Prelude: The Cryptographic Universe and the Great Clique

1.1 Motivation

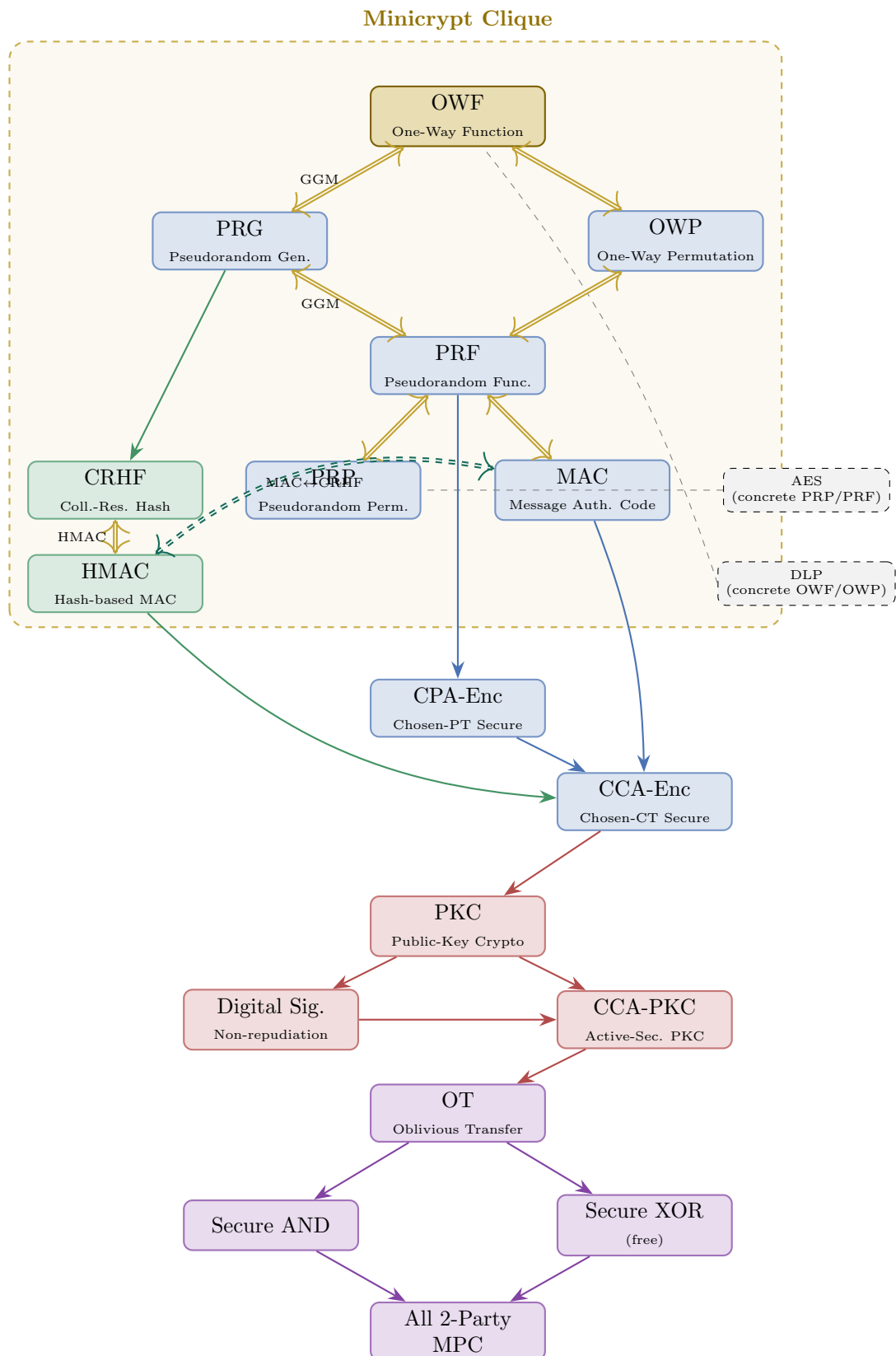
Modern cryptography is built on a beautifully minimal foundation: the existence of a **One-Way Function (OWF)**. From this single computational assumption—that some functions are easy to compute but hard to invert—we can derive *every* symmetric cryptographic primitive used in practice. This chain of reductions is both the theoretical backbone of our assignments and a profound statement about what computation can and cannot do.

1.2 The Cryptographic Primitive Hierarchy

The diagram below shows the *complete reduction graph* among the primitives we study. An arrow $A \rightarrow B$ means “ A implies B ” (i.e., given A we can construct B). The double-headed arrows show the remarkable **equivalences** at the core of Minicrypt.

1.3 Note

We have added comprehensive explanation of each programming assignment to avoid any confusion and assist easy implementations. If you have any doubts, feel free to reach out to your respective TAs.



The Minicrypt Equivalence Theorem

The following are **equivalent** (each implies all others):

- One-Way Functions (OWFs) exist
- Pseudorandom Generators (PRGs) exist
- Pseudorandom Functions (PRFs) exist
- One-Way Permutations (OWPs) exist
- Pseudorandom Permutations (PRPs) exist
- Secure Message Authentication Codes (MACs) exist
- Collision-Resistant Hash Functions (CRHFs) exist (*linked via HMAC*)
- HMAC exists (*bridges CRHF $\leftrightarrow$ MAC*)

This is the *Minicrypt Clique*. Proving any one of these exists immediately gives you all the others—and your assignments will implement every step of this chain. Note that construction of all edges may not be possible since some might be open problems. Try to complete the ones that are already known.

Note on CRHF and HMAC: The classical Minicrypt clique consists of the first six primitives. CRHF and HMAC join the clique through HMAC's security proof: if the hash's compression function is a PRF (which our PA#8 DLP-based hash satisfies), then HMAC is a secure MAC ($CRHF \Rightarrow MAC$). The reverse direction holds because a secure MAC used as a Merkle-Damgrd compression function gives a CRHF ($MAC \Rightarrow CRHF$). This bidirectional link is demonstrated concretely in PA#9 (birthday attack, establishing the security floor of any CRHF) and PA#10 (HMAC construction + CCA-secure encryption).

Implementation Requirement: Bidirectional Reductions (All Clique Primitives)

The equivalences in the Minicrypt Clique are **biconditional**: $A \Leftrightarrow B$, not merely $A \Rightarrow B$.

Rule: You are required to implement a primitive **if and only if** you also implement the reduction in *both* directions for every adjacent pair in the clique. For each pair (A, B) below you must demonstrate: (i) how to *construct* B from A , and (ii) how breaking B would allow you to *break* A (i.e., a reduction in the reverse direction). The complete set of required bidirectional implementations is:

OWF $\Leftrightarrow$ PRG

- *Forward* OWF $\Rightarrow$ PRG: HILL/iterative construction — apply f repeatedly with hard-core predicate b ; output $b(f^0(x)) \| b(f^1(x)) \| \dots$
- *Backward* PRG $\Rightarrow$ OWF: Any PRG G is immediately a OWF. Define $f(s) = G(s)$; inversion of f would invert G , recovering the seed and breaking pseudorandomness.

OWF $\Leftrightarrow$ OWP

- *Forward* OWF $\Rightarrow$ OWP: Any OWF on a domain with efficiently samplable pre-images can be converted to a OWP (e.g., DLP: $f(x) = g^x \bmod p$ is a OWP on $\mathbb{Z}_q$).
- *Backward* OWP $\Rightarrow$ OWF: Immediate — a OWP is a special case of a OWF (bijective, hence also hard to invert).

PRG $\Leftrightarrow$ PRF

- *Forward* PRG $\Rightarrow$ PRF: GGM tree construction. Given $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, define $F_k(b_1 \cdots b_n) = G_{b_n}(\cdots G_{b_1}(k))$.
- *Backward* PRF $\Rightarrow$ PRG: Define $G(s) = F_s(0) \| F_s(1)$. If G were distinguishable from random, the distinguisher could be used to break the PRF.

OWP $\Leftrightarrow$ PRG

- *Forward* OWP $\Rightarrow$ PRG: Any OWP with a hard-core predicate b yields a PRG: $G(x) = (f(x), b(x))$, expanding by one bit per application.
- *Backward* PRG $\Rightarrow$ OWP: A length-preserving PRG is itself a OWP (it must be injective and hard to invert, hence a permutation on its range).

PRF $\Leftrightarrow$ PRP

- *Forward* PRF $\Rightarrow$ PRP: Luby-Rackoff construction. Apply three or four rounds of a Feistel network using the PRF as the round function: a 3-round Feistel yields a secure PRP; a 4-round Feistel yields a secure *strong* PRP (indistinguishable even from an adversary with inversion queries).
- *Backward* PRP $\Rightarrow$ PRF: A PRP over a super-polynomially large domain is computationally indistinguishable from a PRF (by the PRF/PRP switching lemma). Concretely, AES (a PRP) is used directly as a PRF in CTR, OFB, and GCM.

PRF $\Leftrightarrow$ MAC

- *Forward* PRF $\Rightarrow$ MAC: $\text{Mac}_k(m) = F_k(m)$. If the MAC were forgeable, the forger distinguishes F_k from a random function, breaking PRF security.
- *Backward* MAC $\Rightarrow$ PRF: A secure EUF-CMA MAC on uniformly random messages is a PRF. Use the MAC oracle as a PRF oracle; unforgeability implies pseudorandomness of outputs.

PRP $\Leftrightarrow$ MAC (via PRF as bridge)

- *Forward* PRP $\Rightarrow$ MAC: Use the PRP directly as a PRF (switching lemma), then apply PRF $\Rightarrow$ MAC above. Concretely: AES-CMAC and CBC-MAC both use a block cipher (PRP) as the underlying primitive.
- *Backward* MAC $\Rightarrow$ PRP: Via MAC $\Rightarrow$ PRF (above) and then PRF $\Rightarrow$ PRP (Luby-Rackoff).

OWP $\Leftrightarrow$ PRF (completing the clique)

- *Forward* OWP $\Rightarrow$ PRF: OWP $\Rightarrow$ PRG (above) then PRG $\Rightarrow$ PRF (GGM).
- *Backward* PRF $\Rightarrow$ OWP: PRF $\Rightarrow$ PRP (Luby-Rackoff); a PRP on $\{0, 1\}^n$ keyed by k gives OWP $f(k) = \text{PRP}_k(0^n)$.

CRHF $\Leftrightarrow$ HMAC

- *Forward* CRHF $\Rightarrow$ HMAC: Given a hash function H built on a PRF-secure compression function (e.g., your PA#8 DLP hash), define $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m))$. Security holds because the inner hash acts as a PRF on the message, and the outer hash acts as a PRF on the inner hash output.

- *Backward* HMAC $\Rightarrow$ CRHF: Fix a key k and define $H'(m) = \text{HMAC}_k(m)$. This keyed function is collision-resistant (any collision would constitute a MAC forgery). More generally, use the HMAC compression step as the compression function in a new Merkle-Damgrd hash.

HMAC $\Leftrightarrow$ MAC (*HMAC joins the MAC equivalence class*)

- *Forward* HMAC $\Rightarrow$ MAC: HMAC is a secure EUF-CMA MAC when the compression function is a PRF. This is exactly what PA#10 implements.
- *Backward* MAC $\Rightarrow$ HMAC: Any secure PRF-based MAC can be cast in the HMAC double-hash structure by treating the MAC as the inner compression step. The HMAC structure is therefore not a special-case construction but the natural PRF-based MAC for hash-structured primitives.

CRHF $\Leftrightarrow$ MAC (*the full bridge, via HMAC*)

- *Forward* CRHF $\Rightarrow$ MAC: CRHF $\Rightarrow$ HMAC $\Rightarrow$ MAC (two steps above). Concretely: your PA#8 DLP hash $\Rightarrow$ PA#10 HMAC $\Rightarrow$ PA#10 CCA-secure encryption.
- *Backward* MAC $\Rightarrow$ CRHF: A secure MAC serves as a collision-resistant compression function (any collision in the MAC output is a forgery). Apply the Merkle-Damgrd transform (PA#7) to get a full CRHF.

This bidirectionality is what makes these six primitives a *clique*, not merely a chain. Submitting only forward constructions earns partial credit; full credit requires both directions for every pair you implement.

1.4 Concrete Instantiations

In practice we use two primary workhorses:

- **AES (Advanced Encryption Standard):** A concrete, highly optimized *PRP* (and by extension PRF) based on the algebraic structure of $\text{GF}(2^8)$. Whenever an assignment asks for a “generic PRF,” you may plug in AES.
- **Discrete Logarithm Problem (DLP):** The assumption that computing $\log_g h$ in a cyclic group $\mathbb{G}$ is computationally hard. This gives us a concrete OWF/OWP, and underpins Diffie-Hellman, ElGamal, and the DLP-based hash function.

1.5 The No-Library Rule

Global Policy: No External Cryptographic Libraries

Every cryptographic primitive used in any programming assignment **must be your own prior implementation**. You may *not* substitute a library (e.g. PyCryptodome, OpenSSL, BouncyCastle, `cryptography`, Java’s `javax.crypto`, etc.) at any point in the dependency chain.

Specifically:

- If PA#18 (OT) requires a PKC, you must supply your own implementation from

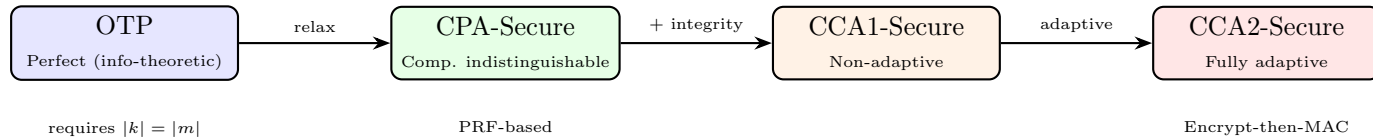
PA#12 or PA#16 — not `import rsa` or equivalent.

- If PA#6 or PA#10 (CCA-Enc) require a MAC, you must supply your own from PA#5 (for PRF-MAC) or PA#10 (for HMAC) — not HMAC from `hashlib` or any other library.
- If PA#10 (HMAC) requires a hash function, you must supply your own `DLP.Hash` from PA#8 — not SHA-256 or any standard library hash.
- If PA#4 (Modes) requires a block cipher, you must supply your own PRF/PRP from PA#2, or your own AES implementation — not a library AES call.
- PA#20 (MPC) must be built end-to-end using your PA#19 (Secure AND) and your PA#18 (OT) — not a garbled-circuit library.

The one permitted exception: You may use standard library functions for *arbitrary-precision integer arithmetic* (e.g. Python's built-in `int`, Java's `BigInteger`) and for *OS-level randomness* (`os.urandom`, `SecureRandom`). Everything above raw integers and random bytes must be your own code.

Why this rule exists: The entire point of these assignments is to trace the reduction chain in code. If you substitute a library primitive at any node, you break the chain and defeat the purpose of the exercise. The dependency graph below makes the required lineage explicit.

1.6 Security Notions Roadmap



2 Part 0: The Minicrypt Clique Web Explorer (Interactive Component)

What This Assignment Is and Why It Exists Here

The programming assignments in Parts I–IV ask you to implement each Minicrypt primitive on the command line. This section adds a complementary deliverable: a **React web application** that makes every reduction in the clique *visual, interactive, and traceable with real data*.

The app does not replace the terminal implementations. It *calls* them. Its purpose is to force you to think about a question the terminal assignments can obscure: **where does the concrete data actually come from?** Every abstract reduction in the clique says “given an oracle for primitive A , construct primitive B .” But an oracle is not code. You have to pick a concrete function—AES or DLP—and trace how that function flows all the way up the chain to B .

Placement: This section is described here, in the Prelude, because the app is built *incrementally* as you complete Parts I–IV. You can stub out primitives you have not yet implemented and fill them in as the course progresses. A fully working version is expected by the final submission deadline.

2.1 The Core Problem the App Must Solve

Consider the reduction $\text{PRG} \Rightarrow \text{PRF}$ (the GGM construction). Abstractly, it says: given any PRG G , define $F_k(x) = G_{x_n}(\dots G_{x_1}(k))$. But to run this on actual data you need an actual PRG. Where does it come from?

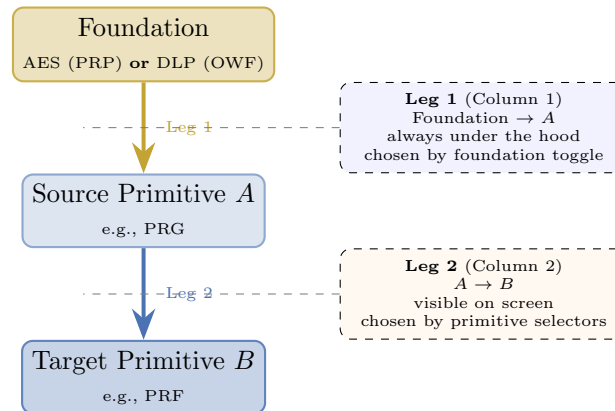
The answer is always one of two concrete starting points:

- **AES** — a concrete PRP/PRF. To get a PRG from AES: use AES as a PRF (PRP/PRF switching lemma), then define $G(s) = F_s(0) \| F_s(1)$.
- **DLP** — a concrete OWF/OWP. To get a PRG from DLP: apply the hard-core-bit construction (HILL), iterating $f(x) = g^x \bmod p$ to extract pseudorandom bits.

In either case, reaching PRF requires **two distinct legs**:

1. **Leg 1 (foundation $\rightarrow$ source primitive):** Instantiate the source primitive from the concrete foundation. This is always done under the hood, regardless of which two clique members the user picks.
2. **Leg 2 (source primitive $\rightarrow$ target primitive):** Apply the abstract reduction. This is the visible operation on screen.

The web app makes both legs explicit, with data flowing through each one.



2.2 Layout Specification

The app has a **fixed three-tier layout**:

1. **Top bar — Foundation selector:** A toggle with two options: AES-128 (PRP) and DLP ($g^x \bmod p$). Changing this toggle affects Column 1 entirely and re-runs all computations.
2. **Two-column main area:**
 - **Column 1 — “Build” panel (Leg 1: Foundation → Source Primitive A):**
 - A dropdown: select the *source primitive* A from the clique (OWF, PRG, PRF, PRP, MAC, CRHF, HMAC).
 - An input field: enter a raw key or seed (hex string).
 - A step-through display: shows each sub-reduction from the foundation to A , with the actual intermediate byte values at each step.
 - Example for AES + PRG: shows $\boxed{\text{AES key}} \xrightarrow{\text{PRF}} \boxed{F_k(0) \| F_k(1)} = \boxed{\text{PRG output}}$.
 - **Column 2 — “Reduce” panel (Leg 2: Source Primitive A → Target Primitive B):**
 - A dropdown: select the *target primitive* B (must differ from A).
 - An input field: enter the message or query to evaluate.
 - A step-through display: shows the reduction from A to B (e.g., GGM tree for PRG→PRF), with real intermediate values.
 - The output of Column 1 is automatically piped as the concrete implementation of A used in Column 2.
3. **Bottom panel — Reduction proof summary:** A collapsible box showing the full chain: Foundation → A → B , the theorem names (HILL, GGM, Luby-Rackoff, etc.), and the security claim at each step.

Foundation: AES-128 (PRP) / DLP ($g^x \bmod p$)
CS8.401 Minicrypt Clique Explorer

Column 1: Build Source Primitive from Foundation

Source primitive A : PRG ▼
Input seed: a3f2... (hex)

 $AES_k(0) = 7f3a...$
 $AES_k(1) = c19b...$
 $PRG(s) = 7f3ac19b...$

Column 2: Reduce Source to Target Primitive

Target primitive B : PRF ▼
Query x : 1011 (bits)

GGM: $G_1(G_0(k))$
 $= G_1(7f3a...)$
 $PRF_k(x) = 88d4...$

Reduction Chain Summary (click to expand)

$AES (PRP) \xrightarrow{\text{switching lemma}} PRF \xrightarrow{G(s)=F_s(0) \parallel F_s(1)} PRG \xrightarrow{\text{GGM tree}} PRF$

Security: PRF-security of AES $\Rightarrow$ PRG-security (negl. advantage) $\Rightarrow$ PRF-security (GGM thm.)

All intermediate values shown above are real outputs from your PA#1–#2 implementations.

2.3 The Under-the-Hood Requirement (Critical)

Architectural Rule: The Foundation Must Flow Through Column 1

Column 2 is **not** allowed to call the foundation (AES or DLP) directly. It may only call the source primitive A that Column 1 has constructed.

Concretely: if the user selects Foundation = AES, Source = PRG, Target = PRF:

- **Correct:** Column 1 builds PRG using AES under the hood. Column 2 receives a PRG object and runs GGM on it, never touching AES directly.
- **Wrong:** Column 2 calls AES directly to implement the GGM tree leaves. This collapses the two legs into one and defeats the purpose.

This rule enforces the reduction structure: each primitive is a black box to the layer above it. The same rule applies to all choices of A and B .

2.4 Supported Reductions and the Routing Table

Not every pair (A, B) has a direct one-step reduction. The app must implement a **routing table** that computes the shortest path in the clique graph and chains the reductions:

Source A	Target B	Reduction chain used
OWF	PRG	HILL hard-core-bit construction
OWF	OWP	DLP is already a OWP (identity for DLP foundation)
PRG	PRF	GGM tree
PRF	PRP	Luby-Rackoff 3-round Feistel
PRF	MAC	$\text{Mac}_k(m) = F_k(m)$
PRP	MAC	PRP/PRF switching lemma, then MAC
CRHF	HMAC	HMAC construction (PA#10)
HMAC	MAC	Direct (HMAC is a MAC)
Any	Any	Compose the above steps as needed

For backward reductions ($B \rightarrow A$), the app also supports the reverse direction (bidirectional mode toggle).

2.5 PA #0 — What You Must Implement

PA #0 — What You Must Implement

1. **React app scaffold:** Create a React app (Create React App or Vite) with the three-tier layout described above. Foundation toggle, two-column main area, and bottom proof panel. Styling is your choice but the layout must be clearly two-column.
2. **Foundation layer:** Implement two foundation modules:
 - **AESFoundation:** wraps your PA#2 AES-based PRF. Exposes `asOWF()`, `asPRF()`, `asPRP()`.
 - **DLPFoundation:** wraps your PA#1 DLP-based OWF. Exposes `asOWF()`, `asOWP()`.

Both modules share a common **Foundation** interface so the rest of the app is agnostic.

3. **Column 1 — Build panel:** Given a Foundation and a target source primitive A , compute and display the full chain $\text{Foundation} \rightarrow A$, showing each intermediate value. Each step must display: the function applied, the input bytes (hex), and the output bytes (hex). The actual computation must call your existing PA implementations compiled to WebAssembly or via a local API — **not** reimplemented in JavaScript.
4. **Column 2 — Reduce panel:** Given the concrete instance of A produced by Column 1 and a target primitive B , compute and display the reduction $A \rightarrow B$ step by step. Column 2 must receive A as a black box (a function object) and must not inspect its internals.
5. **Routing table:** Implement `reduce(A, B, foundation)` that returns the ordered list of reduction steps to chain. Handle all pairs listed in the table above. For unsupported pairs (e.g., $\text{CRHF} \rightarrow \text{OWP}$), display a clear message explaining why no direct path exists in this direction and suggest using the bidirectional toggle.
6. **Live data flow:** When the user changes any input (foundation toggle, source/target primitive, key, message), all panels update in real time without a page reload.
7. **Bidirectional mode:** Add a toggle “Forward ($A \rightarrow B$) / Backward ($B \rightarrow A$)” that swaps the columns and shows the reverse reduction (e.g., $\text{MAC} \rightarrow \text{PRF}$ by querying the MAC as a PRF oracle and running the distinguishing game).
8. **Proof summary panel:** For each selected pair (A, B) , display the formal security chain: which theorem justifies each step, what the security reduction is (“if adversary breaks B with advantage ϵ , it breaks A with advantage $\epsilon' \geq \epsilon/q$ ”), and the PA number

that implements each step.

9. **Stub support:** Primitives you have not yet implemented should show a “Not implemented yet (due: PA#N)” placeholder with a greyed-out step in the chain. The app must be fully runnable with any subset of primitives implemented.

PA #0 — Interactive Demo Deliverable

The scaffold itself *is* the demo for PA#0. Graders will open the app in a browser and check:

- The foundation toggle (AES / DLP) is visible and switches without errors.
- Both columns render with their dropdowns and input fields.
- Selecting any primitive pair shows a “Not yet implemented” placeholder with the correct PA number — not a blank or crash.
- The bottom proof panel opens/closes on click and displays the static reduction chain text for the selected pair.

Toy parameters: No real crypto is needed for PA#0. Stub functions returning fixed hex strings are fine at this stage.

Intuition

Why start from scratch? A truly random One-Time Pad gives *perfect* (Shannon) secrecy but demands a key as long as the message. The key insight of computational cryptography is: we only need an adversary to be *computationally* indistinguishable from random, not information-theoretically so. A short random seed fed into a PRG yields a *pseudorandom* stream that is indistinguishable from true randomness by any efficient adversary. Upgrading to a PRF gives us indexed, non-sequential access, enabling MAC and encryption. This is the bootstrapping miracle of Minicrypt.

2.6 PA #1 — One-Way Functions & Pseudorandom Generators**One-Way Functions (OWFs)****Definition: One-Way Function**

A polynomial-time computable function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is **one-way** if for every PPT adversary $\mathcal{A}$:

$$\Pr [\mathcal{A}(1^n, f(x)) \in f^{-1}(f(x))] \leq \text{negl}(n)$$

where $x \leftarrow \{0, 1\}^n$ uniformly at random.

Concrete OWFs to implement:

- **Modular exponentiation (DLP):** $f(x) = g^x \bmod p$ in a prime-order group.
- **Integer factoring:** $f(p, q) = p \cdot q$ for large primes p, q .
- **AES as a compression OWF:** $f(k) = \text{AES}_k(0^{128}) \oplus k$.

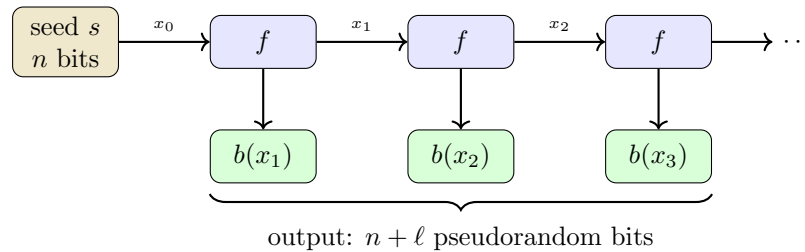
Pseudorandom Generators (PRGs)**Definition: PRG**

A deterministic polynomial-time function $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+\ell(n)}$ (for any $\ell(n) > 0$) is a **PRG** if for every PPT distinguisher D :

$$\left| \Pr_{s \leftarrow \{0, 1\}^n} [D(G(s)) = 1] - \Pr_{r \leftarrow \{0, 1\}^{n+\ell(n)}} [D(r) = 1] \right| \leq \text{negl}(n)$$

Construction of a PRG from a OWF (Håstad–Impagliazzo–Levin–Luby): Given OWF f with hard-core predicate b (e.g., the Goldreich-Levin bit), define:

$$G(x_0) = b(x_0) \parallel b(x_1) \parallel \cdots \parallel b(x_\ell) \quad \text{where } x_{i+1} = f(x_i)$$



Security Claim

If f is a OWF with hard-core predicate b , then G as constructed above is a secure PRG. Security reduces to the hardness of inverting f .

PA #1 — What You Must Implement

1. OWF (choose one concrete instantiation):

- DLP-based: $f(x) = g^x \bmod p$, with a safe prime p and generator g of prime-order subgroup.
- AES-based: $f(k) = \text{AES}_k(0^{128}) \oplus k$ (Davies-Meyer-style).
- Your OWF must expose a `evaluate(x)` function and a `verify_hardness()` demo (show that random inversion fails).

2. PRG from OWF (forward direction, PA#1a):

Implement the iterative hard-core-bit construction. Given seed $s \in \{0,1\}^n$, produce an output of length $n + \ell$ for user-specified ℓ .

3. OWF from PRG (backward direction, PA#1b):

Show that $f(s) = G(s)$ is a OWF. Provide a brief written argument (code comment or README) and a demonstration that an adversary given $G(s)$ cannot recover s efficiently.

4. Statistical test suite:

Run your PRG output through at least three NIST SP 800-22 tests: frequency (monobit), runs, and serial. Report pass/fail and p -values.

5. Interface:

Your PRG must expose `seed(s)`, `next_bits(n)` so that PA#2 can consume it as a black box.

PA #1 — Interactive Demo Deliverable

Demo: Live PRG output viewer. In the web app, PA#1's demo panel shows:

- A hex input field for the seed s (e.g., 16 bytes).
- A slider for output length ℓ (8–256 bytes).
- Live output: as the user types or moves the slider, $G(s)$ updates instantly.
- A “Randomness test” button that runs the frequency and runs tests and displays pass/fail with a bar showing the bit ratio ($\approx 50\%$ ones expected).

Toy parameters: 64-bit seed, DLP group of order $\approx 2^{30}$ or AES-128. Must run in under 1 second.

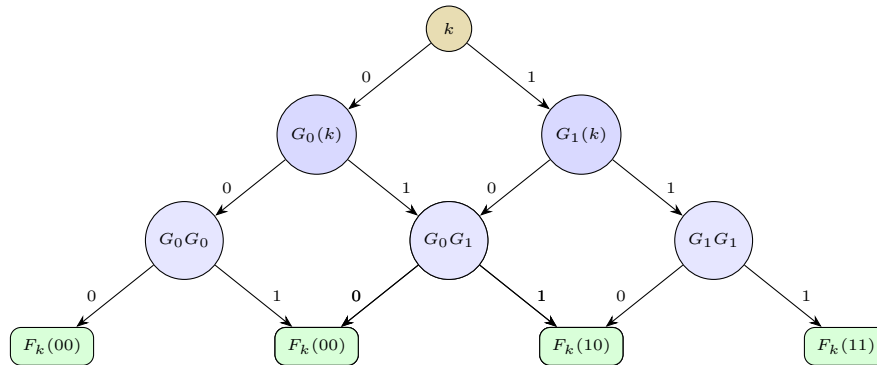
2.7 PA #2 — Pseudorandom Functions via GGM Tree

A PRF $F_k : \{0,1\}^n \rightarrow \{0,1\}^n$ maps an input x to an output that is computationally indistinguishable from a truly random function, when the key k is secret.

The GGM Tree Construction

Given a length-doubling PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, write $G(s) = G_0(s) \| G_1(s)$ (left and right halves). Define:

$$F_k(b_1 b_2 \cdots b_n) = G_{b_n}(G_{b_{n-1}}(\cdots G_{b_1}(k) \cdots))$$



Depth-2 GGM tree for $n = 2$ input bits

PRF from PRG

Inputs: Key $k \in \{0, 1\}^n$, query $x = b_1 \cdots b_n \in \{0, 1\}^n$

Algorithm:

1. Set $s_0 \leftarrow k$
2. For $i = 1$ to n : set $s_i \leftarrow G_{b_i}(s_{i-1})$
3. Output s_n

Cost: n evaluations of G ; tree has 2^n leaves total but only one root-to-leaf path is computed per query.

Plugging in AES: For practical purposes, AES-128 is already a highly efficient PRP/PRF. You may use $F_k(x) = \text{AES}_k(x)$ directly, bypassing the GGM construction.

Security Claim

If G is a secure PRG, then the GGM construction yields a secure PRF. Concretely: any PPT adversary making q queries to F_k cannot distinguish it from a random oracle with advantage greater than $\text{negl}(n)$.

PA #2 — What You Must Implement

1. **GGM PRF from your PA#1 PRG** (forward direction, PA#2a): Implement the depth- n GGM binary tree. Key $k \in \{0, 1\}^n$, input $x \in \{0, 1\}^n$; follow the root-to-leaf path defined by the bits of x , applying G_0 or G_1 at each level. Output the leaf value $F_k(x)$.
2. **PRG from PRF** (backward direction, PA#2b): Implement $G(s) = F_s(0^n) \| F_s(1^n)$ as a length-doubling PRG. Show this produces pseudorandom output by running it through the same statistical test suite as PA#1.
3. **AES plug-in**: Implement an alternative PRF using AES-128 directly: $F_k(x) = \text{AES}_k(x)$ (your own AES or the OS primitive as the one allowed exception). Show that substituting AES for your GGM PRF produces functionally identical results downstream.
4. **Distinguishing game demo**: Write a test that queries your PRF on $q = 100$ random inputs and a truly random function on the same inputs, and confirms no statistical difference (supporting PRF security empirically).
5. **Interface**: Expose $F(k, x)$ so that PA#3, PA#4, and PA#5 can use it as a drop-in.

PA #2 — Interactive Demo Deliverable

Demo: GGM tree visualiser. In the web app:

- Input: key k (hex) and a query x (bit string of length $n \leq 8$).
- Display: the GGM binary tree up to depth n , with each node showing its value in hex. The path taken for query x is highlighted in blue; inactive nodes are greyed out.
- The leaf value is labelled “ $F_k(x) =$ ” and shown in a prominent output box.
- Changing any input bit of x re-highlights the path instantly, showing how a 1-bit change in the query produces an uncorrelated output.

Toy parameters: $n = 4$ – 8 bits (tree depth), so the diagram stays readable. AES or your PRG as leaves.

2.8 PA #3 — CPA-Secure Symmetric Encryption**Intuition**

Chosen-Plaintext Attack (CPA): The adversary can encrypt arbitrary messages of their choice (e.g., by sending them to an encryption oracle). CPA security demands that the adversary cannot learn anything about which of two equal-length messages was encrypted, even after seeing polynomially many encryptions.

Standard CPA-Secure Construction

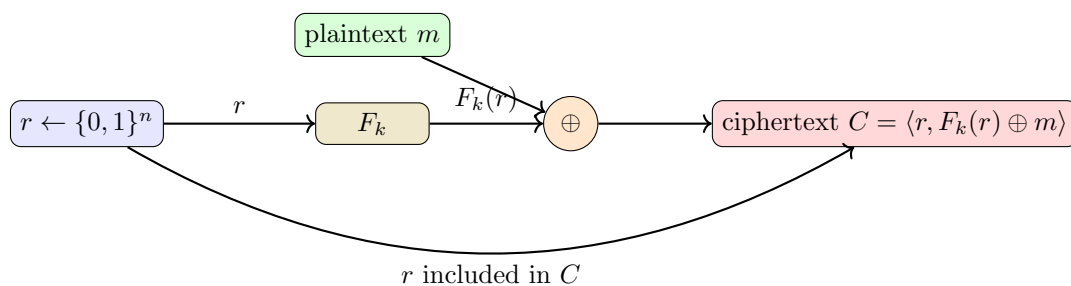
Enc-then-PRF Construction

Key: $k \in \{0, 1\}^n$ **Encryption** of $m \in \{0, 1\}^n$:

1. Sample $r \leftarrow \{0, 1\}^n$ uniformly at random (fresh each encryption).
2. Output $C = \langle r, F_k(r) \oplus m \rangle$

Decryption of $C = \langle r, c \rangle$:

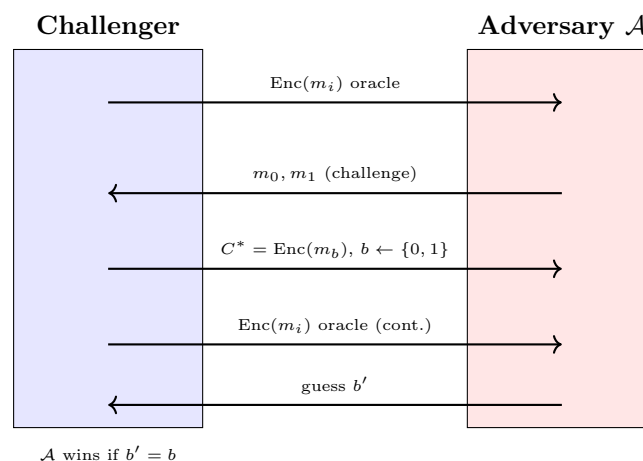
1. Compute $m = F_k(r) \oplus c$



Security Claim

The scheme above is CPA-secure if F_k is a secure PRF. The key is that r is *fresh and random* for each encryption; reusing r breaks security catastrophically.

CPA Security Game



PA #3 — What You Must Implement

1. **CPA-secure encryption scheme:** Implement $\text{Enc}(k, m)$ and $\text{Dec}(k, c)$ using the construction $C = \langle r, F_k(r) \oplus m \rangle$ where r is freshly sampled each call. Your F_k must be your own PRF from PA#2 (not a library).

2. **Multi-block support:** Extend to messages longer than one block by applying the PRF to $r, r+1, r+2, \dots$ (counter-based extension). Correctly handle messages whose length is not a multiple of the block size (padding).
3. **CPA game simulation:** Implement the IND-CPA game. Run it with a dummy adversary that queries the encryption oracle 50 times and then attempts to distinguish — confirm their advantage is ≈ 0 .
4. **Broken variant (demonstrate the attack):** Implement a version that reuses r (i.e., deterministic encryption). Show that this version is *broken* by an adversary that queries two equal messages and detects the identical ciphertexts.
5. **Interface:** Expose $\text{Enc}(k, m) \rightarrow (r, c)$ and $\text{Dec}(k, r, c) \rightarrow m$ for use in PA#6.

PA #3 — Interactive Demo Deliverable

Demo: Play the IND-CPA game. The student acts as the adversary:

- Step 1: Student types two messages m_0 and m_1 of equal length.
- Step 2: Click “Encrypt.” The challenger picks a random bit b and shows $C^* = \text{Enc}_k(m_b)$.
- Step 3: Student guesses b . The app reveals whether the guess was correct and updates a running advantage counter over multiple rounds (should converge to ≈ 0).
- Bonus: a “Reuse nonce” toggle breaks security — with nonce reuse, a correct guess is trivially forced and the advantage counter jumps to 1.

Run 20 rounds; confirm advantage ≤ 0.1 in secure mode and $= 1.0$ in broken mode.

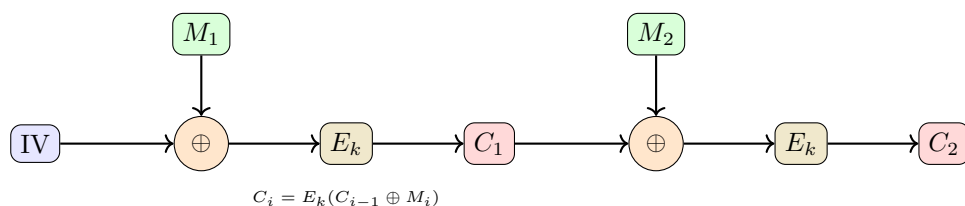
2.9 PA #4 — Modes of Operation

Intuition

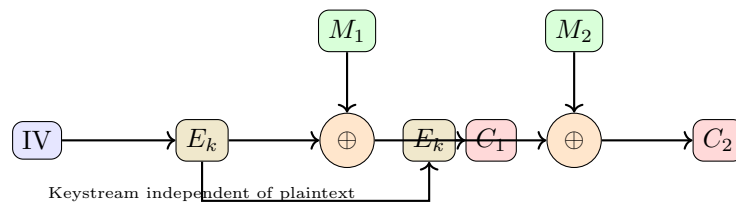
Block ciphers operate on fixed-size blocks (e.g., 128-bit for AES). Real messages are longer. *Modes of operation* specify how to chain block cipher evaluations to handle arbitrary-length messages securely. Different modes offer different trade-offs in parallelism, error propagation, and security properties.

You must support:

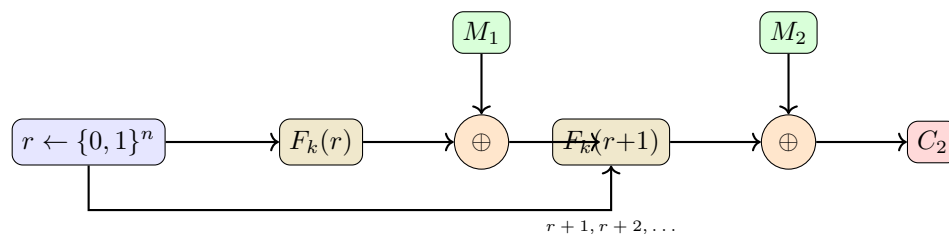
Mode 1: CBC (Cipher Block Chaining)



Properties: Sequential encryption (cannot parallelize encryption), parallel decryption, one-block error propagation. Requires random IV.

Mode 2: OFB (Output Feedback)

Properties: Pre-computable keystream; decryption identical to encryption. Keystream never reuses state. Bit flips in ciphertext produce bit flips in plaintext (no error propagation).

Mode 3: Randomized CTR (Counter Mode)

Properties: Fully parallelizable (both encryption and decryption), turns block cipher into stream cipher. The nonce r must be unique per key.

Comparison Table

Mode	Parallel Enc.	Parallel Dec.	Random Access	Error Prop.	IV Reuse
CBC		✓		2 blocks	Fatal
OFB				None	Fatal
CTR	✓	✓	✓	None	Fatal

PA #4 — What You Must Implement

- CBC mode:** Implement `CBC.Enc(k, IV, M)` and `CBC.Dec(k, IV, C)` for arbitrary-length messages. The IV must be randomly generated per encryption call. Use your own PRF/PRP from PA#2 (or your own AES) as the block cipher E_k .
- OFB mode:** Implement `OFB.Enc(k, IV, M)` and `OFB.Dec(k, IV, C)`. Confirm that encryption and decryption are identical operations. Demonstrate that the keystream can be pre-computed before the plaintext is known.
- Randomized CTR mode:** Implement `CTR.Enc(k, M)` (samples random nonce r internally) and `CTR.Dec(k, r, C)`. Demonstrate parallel block computation.
- Mode selector interface:** Expose a unified API `Encrypt(mode, k, M)` and `Decrypt(mode, k, C)` where `mode` $\in$ {CBC, OFB, CTR}. Internally routes to the correct implementation.
- Attack demos:** (a) Show CBC IV-reuse attack: encrypt two different messages with the same IV and demonstrate that block i leaks if $M_i = M'_i$. (b) Show OFB keystream-reuse attack: encrypt two messages with the same IV and XOR the ciphertexts to recover plaintext XOR.

6. **Correctness tests:** For each mode, verify $\text{Dec}(k, \text{Enc}(k, M)) = M$ on at least three message lengths: shorter than one block, exactly one block, and spanning multiple blocks.

PA #4 — Interactive Demo Deliverable

Demo: Block cipher mode animator. For a 3-block message:

- Three mode tabs: CBC, OFB, CTR. Switching tabs re-runs the animation.
- Each block is a coloured box. Arrows animate in sequence showing XOR operations and block cipher calls, with each intermediate value shown in hex.
- “Flip bit” button: student clicks any ciphertext block; the app re-decrypts and highlights which plaintext blocks are corrupted (CBC: 2 blocks; OFB: only the same block; CTR: only the same block).
- “Reuse IV” toggle: re-encrypts two messages with the same IV in CBC mode and highlights the matching ciphertext blocks in red.

Flip a ciphertext bit in each mode and verify correct error propagation pattern.

2.10 PA #5 — Message Authentication Codes (MACs)

Intuition

The need for integrity: Encryption provides *confidentiality* but not *integrity*. An adversary can flip bits in a ciphertext and the decryption will produce a modified (but valid-looking) plaintext. A MAC allows a receiver to verify that a message was not tampered with, without requiring a public key.

MAC Definition

A MAC scheme $(\text{Mac}, \text{Vrfy})$ with key space $\mathcal{K}$, message space $\mathcal{M}$, and tag space $\mathcal{T}$ satisfies:

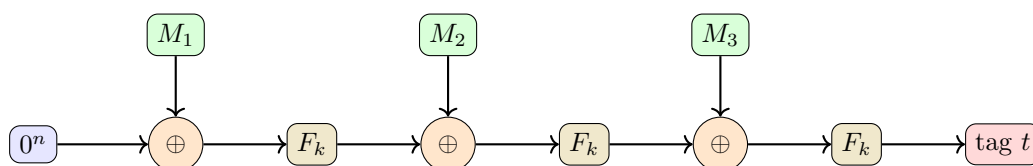
- $\text{Mac}_k(m) \rightarrow t$: produce tag t for message m under key k .
- $\text{Vrfy}_k(m, t) \rightarrow \{0, 1\}$: verify that t is a valid tag for m .

Security (EUF-CMA): No PPT adversary making polynomially many $(m_i, \text{Mac}_k(m_i))$ queries can forge a valid tag on a *new* message.

Three constructions to implement:

Construction 1: PRF-MAC (Fixed-length) For messages $m \in \{0, 1\}^n$: set $t = F_k(m)$.

Construction 2: CBC-MAC (Variable-length)



Construction 3: HMAC Using a hash function H : $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m))$.

PA #5 — What You Must Implement

1. **PRF-MAC (fixed-length)**: Implement $\text{Mac}(k, m) = F_k(m)$ and $\text{Vrfy}(k, m, t)$. The PRF F_k must be your own from PA#2. This handles messages of exactly one block length n .
2. **CBC-MAC (variable-length)**: Extend to arbitrary-length messages using the CBC-MAC construction (chain F_k over message blocks, output the final chaining value as the tag). Implement both Mac and Vrfy .
3. **HMAC (forward pointer)**: PA#5 introduces the HMAC formula for completeness, but the full implementation of HMAC belongs in **PA#10**. In PA#5, you may leave a clearly labelled stub function $\text{hmac}(k, m)$ that raises `NotImplemented` until PA#10 is complete. *Do not use a library HMAC here or in PA#10.*
4. **MAC $\Rightarrow$ PRF (backward direction)**: Demonstrate that your PRF-MAC, when queried on uniformly random inputs, passes the same PRF distinguishing test from PA#2. This concretely witnesses $\text{MAC} \Rightarrow \text{PRF}$.
5. **EUFCMA forgery demo**: Implement the EUFCMA game. Show that a naive adversary (who has seen up to 50 (m_i, t_i) pairs) cannot produce a valid tag on a new message. Separately, demonstrate a *length-extension attack* on a naive single-hash MAC ($t = H(k \| m)$) to motivate the HMAC double-hash structure.
6. **Interface**: Expose $\text{Mac}(k, m) \rightarrow t$ and $\text{Vrfy}(k, m, t) \rightarrow \text{bool}$ for use in PA#6.

PA #5 — Interactive Demo Deliverable

Demo: MAC forge attempt. The student acts as the EUFCMA adversary:

- The app shows a running list of up to 50 signed messages (m_i, t_i) generated by a hidden key k .
- Student types a new message m^* (not in the list) and a tag t^* .
- Click “Submit forgery.” The app verifies and shows “Forgery accepted” or “Forgery rejected.”
- A counter tracks forgery attempts and successes. Graders expect 0 successes in ≥ 20 attempts.
- Separate tab: “Length-extension demo” — student types a suffix m' , app shows the extended tag on $m \| \text{pad} \| m'$ being computed from t alone (without k), demonstrating the naive $H(k \| m)$ vulnerability.

2.11 PA #6 — CCA-Secure Symmetric Encryption

Intuition

Why CPA is not enough: CPA-secure schemes are *malleable*—an adversary can modify a ciphertext in a predictable way to alter the plaintext without knowing the key. CCA (Chosen-Ciphertext Attack) security eliminates this by requiring that even an adversary with a decryption oracle cannot break security. The **Encrypt-then-MAC** paradigm achieves this.

Encrypt-then-MAC Construction

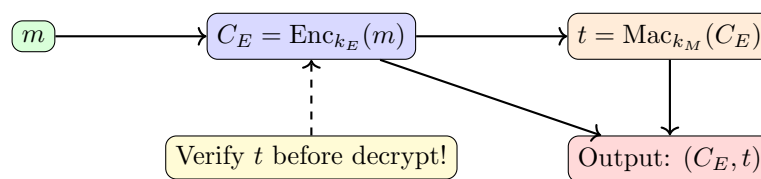
Keys: Independent encryption key k_E and MAC key k_M .

Encryption of m :

1. $C_E \leftarrow \text{Enc}_{k_E}(m)$ (CPA-secure, e.g., PA#3)
2. $t \leftarrow \text{Mac}_{k_M}(C_E)$ (EUF-CMA secure, e.g., PA#5)
3. Output (C_E, t)

Decryption of (C_E, t) :

1. If $\text{Vrfy}_{k_M}(C_E, t) = 0$: output $\perp$ (reject tampered ciphertext)
2. Else: output $\text{Dec}_{k_E}(C_E)$



MAC check prevents CCA oracle exploitation

Security Claim

If Enc is CPA-secure and Mac is EUF-CMA secure, then Encrypt-then-MAC is **CCA2-secure**. The MAC verification step means any adversary query to the decryption oracle on a modified ciphertext is rejected, nullifying the adaptive oracle.

PA #6 — What You Must Implement

1. **Encrypt-then-MAC:** Implement $\text{CCA_Enc}(k_E, k_M, m)$ and $\text{CCA_Dec}(k_E, k_M, c, t)$. Internally, CCA_Enc calls your PA#3 Enc and your PA#5 Mac. CCA_Dec must call *Vrfy* before calling *Dec* — rejecting with $\perp$ on MAC failure.
2. **Key separation:** Use independently sampled keys k_E and k_M (never the same key for both). Demonstrate that reusing a single key for both roles creates exploitable correlations.
3. **CCA2 game simulation:** Implement the IND-CCA2 game. Your adversary gets both an encryption oracle and a decryption oracle (which rejects the challenge ciphertext). Show that your scheme's adversary advantage is ≈ 0 .
4. **Malleability attack on CPA-only:** Show that your PA#3 CPA-secure scheme (without the MAC) is *malleable*: given $C = \langle r, F_k(r) \oplus m \rangle$, an adversary can flip bit i of the ciphertext to produce a ciphertext for m with bit i flipped, without knowing k or m . Contrast this with your CCA-secure scheme where the same attack is detected and rejected.
5. **Interface:** Expose $\text{CCA_Enc}(k_E, k_M, m) \rightarrow (c, t)$ and $\text{CCA_Dec}(k_E, k_M, c, t) \rightarrow m$ or $\perp$.

PA #6 — Interactive Demo Deliverable

Demo: Malleability attack panel.

- Left side (CPA-only): student is shown a ciphertext $C = \langle r, F_k(r) \oplus m \rangle$. A bit-flip

tool lets them click any bit. The app decrypts the modified ciphertext and shows the corrupted plaintext — demonstrating malleability.

- Right side (CCA / Encrypt-then-MAC): the same bit-flip attempt is shown, but the MAC verification step fires and returns $\perp$. The plaintext never decrypts.
- Both sides update live as the student flips different bits.

Flip a bit on the left; confirm modified plaintext appears. Flip the same bit on the right; confirm $\perp$ is returned.